

深圳大学实验报告

课程名称：计算机系统 1-组成概论

实验项目名称：实验 4：LC-3 简单游戏设计

学院：计算机与软件学院

专业：计算机类

指导教师：李庚辉

报告人：吴淇钺 学号：2025150058 班级：计算机类 02

实验时间：2026 年 5 月 27 日-2026 年 6 月 24 日

实验报告提交时间：2026 年 6 月 24 日

教务部制

实验内容与要求：

请按着试验材料完成实验内容和要求

一．整体设计思路

游戏状态可以用三个计数器和当前玩家表示：

- `ROW\_A`，`ROW\_B`，`ROW\_C` 分别存放每行石头数，初值 3、5、8
- `CUR\_PLAYER` 记录当前回合玩家（1 或 2）
- 通过 `PRINT\_BOARD` 显示界面，`GET\_VALID\_MOVE` 循环读取并校验输入，主循环更新状态、检查胜利、切换玩家

寄存器使用约定

- `R0` / `R1`：TRAP 调用参数、临时数据，子程序返回值
- `R2` ~ `R5`：临时计算，用前保存、用完恢复
- `R6`：暂未使用
- `R7`：存放 JSR 返回地址（每个子程序必须保存）

子程序清单

子程序	功能
----- -----	
`PRINT_BOARD`	输出三行 “ROW X: ooo...”
`GET_VALID_MOVE`	循环提示并读入两个字符，返回合法行索引和数量
主程序	初始化、调度、胜利判定

输入用 `GETC` 回显用 `OUT`，每次输入后输出换行，无效则重新提示同一玩家。

- 石头数量用 `ROW\_A`，`ROW\_B`，`ROW\_C` 三个连续单元模拟数组，通过 `LEA R1, ROW\_A` 然后 `ADD R1, R1, R3` 即可用行索引访问。

`CUR\_PLAYER` 始终存 1 或 2，切换时用 `3 - player` 的小技巧，避免复杂分支。

`GET\_VALID\_MOVE` 是整个游戏的核心难点，必须处理所有非法输入：

- 无论第一个字符是否合法，都会读入第二个字符并回显，完全符合实验要求（如 `&4` 会先回显 `&` 再回显 `4`）。
- 行字母检查：`char - 'A'` 必须在 0~2 之间。
- 数字检查：`digit - '0'` 必须 1~8，且 ****不能超过该行剩余石头数****，否则提示 `Invalid move. Try again.` 并重新提示同一玩家。
- 每次无效后都会重新输出 `Player X, choose...`，与样例输出一致。
- `ROW A: ooo` 中冒号后有一个空格，石头是小写字母 `o`，每行末尾一个换行。
- 当某行石头数变为 0（如 `ROW B:`）时，只输出标签和换行，不会输出多余的 `o`。
- 获胜信息带换行，之后执行 `HALT`。

二．完整 LC-3 汇编代码及相关的注释如下：

```
.ORIG x3000
```

```
=====
; 主程序 MAIN
=====
MAIN
```

```
    ; 初始化石头数: A=3, B=5, C=8
```

```
    LEA    R0, ROW_A
```

```
    AND    R1, R1, #0
```

```
ADD      R1, R1, #3
STR      R1, R0, #0          ; ROW_A = 3
ADD      R1, R1, #2
STR      R1, R0, #1          ; ROW_B = 5
ADD      R1, R1, #3
STR      R1, R0, #2          ; ROW_C = 8
```

； 当前玩家设为 1

```
AND      R1, R1, #0
ADD      R1, R1, #1
ST       R1, CUR_PLAYER
```

； 首次显示棋盘

```
JSR      PRINT_BOARD
```

GAMELOOP

```
JSR      GET_VALID_MOVE      ; 返回 R0=行索引 (0/1/2), R1=数量
```

； 更新对应行的石头数

```
LEA      R2, ROW_A
ADD      R2, R2, R0          ; R2 = &ROW_A + row
LDR      R3, R2, #0          ; 原有数量
NOT      R1, R1
ADD      R1, R1, #1          ; R1 = -quantity
ADD      R3, R3, R1          ; 新数量
STR      R3, R2, #0
```

； 检查是否所有石头为 0（胜利条件）

```
LEA      R4, ROW_A
LDR      R0, R4, #0
LDR      R1, R4, #1
ADD      R0, R0, R1
LDR      R1, R4, #2
ADD      R0, R0, R1
BRz      WIN
```

； 未胜利：切换玩家（3 - CUR_PLAYER）

```
LD       R0, CUR_PLAYER
NOT      R0, R0
ADD      R0, R0, #1
ADD      R0, R0, #3
ST       R0, CUR_PLAYER
```

； 显示更新后的棋盘

```

        JSR        PRINT_BOARD
        BRnzp     GAMELOOP

WIN
        ; 根据 CUR_PLAYER 输出获胜信息
        LD        R0, CUR_PLAYER
        ADD       R0, R0, #-1
        BRz       WIN_P1
        LEA       R0, WIN2_STR
        BR        PUT_WIN
WIN_P1  LEA       R0, WIN1_STR
PUT_WIN TRAP      x22
        HALT

;=====
; 数据区 (靠近 MAIN)
;=====
ROW_A   .BLKW    1    ; 实际用连续 3 个单元作为数组
ROW_B   .BLKW    1
ROW_C   .BLKW    1
CUR_PLAYER .BLKW 1

WIN1_STR .STRINGZ "Player 1 Wins.\n"
WIN2_STR .STRINGZ "Player 2 Wins.\n"

;=====
; 子程序 GET_VALID_MOVE
; 功能: 反复提示当前玩家, 直到输入合法字母(A/B/C)+数字(≤剩余数)
; 返回: R0 = 行索引 (0~2), R1 = 移除数量
;=====
GET_VALID_MOVE
        ; 保存寄存器 (R2~R5, R7)
        ST        R7, GM_SAVE_R7
        ST        R2, GM_SAVE_R2
        ST        R3, GM_SAVE_R3
        ST        R4, GM_SAVE_R4
        ST        R5, GM_SAVE_R5

GM_LOOP
        ; 根据 CUR_PLAYER 输出提示
        LD        R0, CUR_PLAYER
        ADD       R0, R0, #-1
        BRz       GM_P1
        LEA       R0, GM_PROMPT2

```

```

BR          GM_PUTS
GM_P1      LEA      R0, GM_PROMPT1
GM_PUTS    TRAP      x22

; 读第一个字符 (行字母)
TRAP      x20
TRAP      x21
ADD       R2, R0, #0          ; R2 = 行字母

; 读第二个字符 (数字)
TRAP      x20
TRAP      x21
ADD       R3, R0, #0          ; R3 = 数字字符

; 输出换行
LD        R0, GM_NEWLINE
TRAP      x21

; ----- 验证行字母 (必须为 A/B/C) -----
LD        R4, GM_CHAR_A
NOT       R4, R4
ADD       R4, R4, #1          ; -'A'
ADD       R0, R2, R4          ; R0 = char - 'A'
BRn       GM_INVALID
ADD       R1, R0, #-2
BRp       GM_INVALID          ; 不是 0~2 则无效
ADD       R5, R0, #0          ; R5 = 行索引 (0/1/2)

; ----- 验证数字字符 (1~8, 且 ≤ 该行石头数) -----
LD        R4, GM_CHAR_0
NOT       R4, R4
ADD       R4, R4, #1          ; -'0'
ADD       R0, R3, R4          ; R0 = 数字值
ADD       R1, R0, #-1
BRn       GM_INVALID          ; <1
ADD       R1, R0, #-8
BRp       GM_INVALID          ; >8

; 检查数字是否 ≤ 该行剩余数量
LEA       R1, ROW_A
ADD       R1, R1, R5          ; R1 = &ROW_A[row]
LDR       R2, R1, #0          ; R2 = 当前石头数
NOT       R2, R2
ADD       R2, R2, #1          ; -count

```

```

ADD      R2, R0, R2          ; number - count
BRp      GM_INVALID          ; number > count 无效

```

； 合法： 设置返回值和恢复寄存器

```

ADD      R1, R0, #0          ; R1 = 数量
ADD      R0, R5, #0          ; R0 = 行索引
LD       R2, GM_SAVE_R2
LD       R3, GM_SAVE_R3
LD       R4, GM_SAVE_R4
LD       R5, GM_SAVE_R5
LD       R7, GM_SAVE_R7
RET

```

GM_INVALID

```

LEA      R0, GM_ERR_MSG
TRAP     x22
BRnzp    GM_LOOP

```

； GET_VALID_MOVE 使用的局部数据

```

GM_SAVE_R7 .BLKW 1
GM_SAVE_R2 .BLKW 1
GM_SAVE_R3 .BLKW 1
GM_SAVE_R4 .BLKW 1
GM_SAVE_R5 .BLKW 1
GM_CHAR_A  .FILL x0041      ; 'A'
GM_CHAR_0  .FILL x0030      ; '0'
GM_NEWLINE .FILL x000A
GM_PROMPT1 .STRINGZ "Player 1, choose a row and number of rocks: "
GM_PROMPT2 .STRINGZ "Player 2, choose a row and number of rocks: "
GM_ERR_MSG .STRINGZ "Invalid move. Try again.\n"

```

；=====

； 子程序 PRINT_BOARD

； 功能： 输出三行石头状态（若某行数量为 0， 则只输出 ROW X: 和换行）

；=====

PRINT_BOARD

```

ST       R7, PB_SAVE_R7
ST       R0, PB_SAVE_R0
ST       R1, PB_SAVE_R1
ST       R2, PB_SAVE_R2
ST       R3, PB_SAVE_R3
ST       R4, PB_SAVE_R4

```

```

AND      R3, R3, #0          ; R3 = 行索引 i = 0

```

PB_LOOP		ADD	R4, R3, #-3	
		BRzp	PB_DONE	
				; 输出行标签 "ROW A:" / "ROW B:" / "ROW C:"
		ADD	R0, R3, #0	
		BRz	PB_A	
		ADD	R0, R3, #-1	
		BRz	PB_B	
		LEA	R0, PB_ROW_C	
		BR	PB_PUTS	
PB_A	LEA	R0, PB_ROW_A		
	BR	PB_PUTS		
PB_B	LEA	R0, PB_ROW_B		
PB_PUTS	TRAP	x22		
				; 取该行石头数量
	LEA	R1, ROW_A		
	ADD	R1, R1, R3		
	LDR	R2, R1, #0		; R2 = count
				; 输出 count 个 'o'
PB_O_LOOP		ADD	R2, R2, #0	
		BRz	PB_NL	; 修改: 跳转到输出换行的指令
标签				
	LD	R0, PB_CHAR_O		
	TRAP	x21		
	ADD	R2, R2, #-1		
	BRnzp	PB_O_LOOP		
PB_NL				; 新标签名, 避免与数据标签
冲突				
	LD	R0, PB_NEWLINE		
	TRAP	x21		
	ADD	R3, R3, #1		
	BRnzp	PB_LOOP		
PB_DONE				
	LD	R0, PB_SAVE_R0		
	LD	R1, PB_SAVE_R1		
	LD	R2, PB_SAVE_R2		
	LD	R3, PB_SAVE_R3		
	LD	R4, PB_SAVE_R4		

```
LD      R7, PB_SAVE_R7
RET
```

； PRINT_BOARD 使用的局部数据

```
PB_SAVE_R7 .BLKW 1
PB_SAVE_R0 .BLKW 1
PB_SAVE_R1 .BLKW 1
PB_SAVE_R2 .BLKW 1
PB_SAVE_R3 .BLKW 1
PB_SAVE_R4 .BLKW 1
PB_CHAR_0  .FILL x006F      ; 'o'
PB_NEWLINE .FILL x000A
PB_ROW_A   .STRINGZ "ROW A: "
PB_ROW_B   .STRINGZ "ROW B: "
PB_ROW_C   .STRINGZ "ROW C: "
.END
```

将代码输入到 LC3 编辑器后汇编结束的效果如图



三. 样例测试

想要输入的样例和预期的效果思路如下：

- `B2` → 更新 B 行
- `A1` → 减少 A 行
- `C6` → 减少 C 行
- `G1` → 无效，重复提示
- `B3` → 清空 B 行（输出 `ROW B:` 空行）
- `A3` → 超量，无效
- `C2` → 清空 C 行
- `A1` → 减少 A 行
- `A1` → 最后一块石头，玩家 1 获胜

实验实际效果如下：

LC3 Console

ROW A: ooo
ROW B: ooooo
ROW C: ooooooooo
Player 1, choose a row and number of rocks: B2
ROW A: ooo
ROW B: ooo
ROW C: ooooooooo
Player 2, choose a row and number of rocks: A1
ROW A: oo
ROW B: ooo
ROW C: ooooooooo
Player 1, choose a row and number of rocks: C6
ROW A: oo
ROW B: ooo
ROW C: oo
Player 2, choose a row and number of rocks: G1
Invalid move. Try again.
Player 2, choose a row and number of rocks: B3
ROW A: oo
ROW B:
ROW C: oo
Player 1, choose a row and number of rocks:
Player 1 Wins.

LC3 Console

ROW A: oo
ROW B: ooo
ROW C: oo
Player 2, choose a row and number of rocks: G1
Invalid move. Try again.
Player 2, choose a row and number of rocks: B3
ROW A: oo
ROW B:
ROW C: oo
Player 1, choose a row and number of rocks: A3
Invalid move. Try again.
Player 1, choose a row and number of rocks: C2
ROW A: oo
ROW B:
ROW C:
Player 2, choose a row and number of rocks: A1
ROW A: o
ROW B:
ROW C:
Player 1, choose a row and number of rocks: A1
Player 1 Wins.
----- Halting the processor -----

实验结果符合预期，代码符合要求

四. 实验结论

本次实验基于 LC-3 汇编语言完成了取石子小游戏的设计、编码与调试，顺利达成全部实验要求。程序实现了棋盘展示、玩家交替操作、输入合法性校验、游戏状态更新以及胜负判定等核心功能，经过多组正常输入、非法输入、边界场景测试，运行结果与预期完全一致，整体逻辑稳定可靠。在编码过程中，我熟练掌握了 LC-3 常用指令、TRAP 系统调用以及子程序的编写规范，深刻理解了寄存器现场保护与恢复的重要性，尤其是子程序调用时 R7 返回地址的保存操作，保障了程序正常跳转执行。同时，借助连续内存单元模拟数组访问游戏数据，运用数值运算技巧实现玩家切换，简化了分支逻辑，提升了代码简洁度。

此次实验也让我体会到汇编语言底层编程的特点，其代码逻辑与硬件运行联系紧密，对流程梳理、细节把控和排错能力要求较高。后续我会继续巩固 LC-3 汇编知识，强化模块化编程思维，提升底层程序设计与调试能力

说明：

(1) 本次实验课作业满分为 100 分。

(2) 本次实验课作业截至时间 2026 年 6 月 24 日 23:59。

(3) 报告正文：请在指定位置填写，本次实验需要提交源程序文件（实验报告和源程序打包提交）。

(4) 个人信息：WORD 文件名中的“姓名”、“学号”，请改为你的姓名和学号；

实验报告的首页，请准确填写“学院”、“专业”、“报告人”、“学号”、“班级”、“实验报告提交时间”等信息。

(5) 提交方式：请在畅课平台中提交；

(6) 发现抄袭（包括复制&粘贴整句话、整张图），该次作业记零分。

(7) 延迟提交时间超过 1 周，不得分。

(8) 期末考试阶段补交无效。

(9) 实验报告命名方式为：学号-姓名-实验 4.pdf



深圳大学学生实验报告用纸

指导教师批阅意见：
成绩评定：
指导教师签字：
2026 年 6 月 25 日
备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 7 日内。